

Mini Competition: A Small Generative Model for Image Denoising

1. Overview

Design and train a generative model that denoises images and removes small artifacts, at a fixed input and output resolution of 256×256 . Your model should learn a general-purpose denoising capability, not merely memorize how to reverse a small, fixed set of corruption functions. Your model will be evaluated on a diverse, held-out set of images and degradations, which are not disclosed in advance.

This task is intentionally framed as a foundation-model problem: the goal is a model that generalizes across image content and degradation type, rather than one narrowly tuned to a single denoising scenario.

2. Task Definition

- Input: a single corrupted RGB image of size $256 \times 256 \times 3$.
- Output: a single denoised RGB image of size $256 \times 256 \times 3$ (same spatial dimensions as the input). If your model output is not uint8, your code must have provision to convert it to uint8 (intensity range $[0, 255]$). The final output image from your code should be a unit8 (intensity range $[0, 255]$) image.

3. Training Setup

3.1 Training Data

You are free to use any images for training, provided you have the right to use them (e.g., public-domain, Creative Commons, or standard research benchmark datasets). You are responsible for generating/ collecting your own corrupted/clean training pairs.

3.2 Degradations You May Train Against

You may train your model against any combination and severity of the following common degradation types:

- Gaussian noise
- Poisson noise
- Salt-and-pepper noise
- Speckle noise
- Gaussian blur
- Motion blur
- Defocus blur
- JPEG compression artifacts
- Chromatic aberration
- Mixed/combined degradations (e.g., noise and blur applied together)

This list is illustrative of the degradation families your model should be robust to. **However, the held-out set may contain other types of noise/ corruption.** Therefore, you are encouraged to train against a broad and varied set of corruptions and severities rather than a narrow selection.

3.3 Compute Budget

There is no fixed compute budget for training. Your choice of model size and training duration is entirely your own. This trade-off is itself part of what is being evaluated (see Section 5, FLOPs).

4. Held-Out Evaluation Set

Your model will be evaluated on a private held-out set of approximately 100 diverse images, each corrupted with degradations and severities that are not disclosed in advance. The held-out set may include, but is not limited to, degradations such as Gaussian noise, Poisson noise, salt-and-pepper noise, speckle noise, Gaussian blur, motion blur, defocus blur, JPEG compression artifacts, chromatic aberration, and combinations thereof, at varying severities.

The held-out set spans diverse content domains, including natural photography, low-light imagery, and medical imaging (e.g., X-ray, MRI, etc.). Exact image content, degradation types, and severities used for evaluation are not shared with participants.

5. Evaluation Metrics

Submissions will be evaluated centrally, using performance metrics listed below on the held-out set.

Axis	Type	Reference Implementation	Direction
FLOPs	Efficiency	<code>fvcore / ptflops</code> , measured centrally at 256x256	Lower is better
PSNR	Full-reference, pixel-level	<code>skimage.metrics.peak_signal_noise_ratio</code>	Higher is better
SSIM	Full-reference, structural	<code>skimage.metrics.structural_similarity</code>	Higher is better
DISTS	Full-reference, perceptual	DISTS-pytorch (official implementation)	Lower is better

Notes on evaluation:

- FLOPs will be measured on a fixed input tensor of shape (1, 3, 256, 256).
- PSNR, SSIM, and DISTS are full-reference metrics, computed against the private ground-truth clean image for each held-out example, and averaged across the held-out set.
- All full-reference metrics are computed on images represented as uint8 in the [0, 255] range, at the fixed 256x256 resolution.

6. Scoring and Ranking

Each of the four evaluation axes: FLOPs, PSNR, SSIM, and DISTS, is ranked independently across all submissions, from best (rank 1) to worst, using the “lower/higher is better” direction specified in Section 5.

Your final competition score will be dependent on all the metrics.

Because all axes are weighted in this scheme, being decisively best on a single metric is worth exactly as much as being marginally best on it. You are encouraged to consider this when deciding whether to optimize heavily for one axis or pursue a more balanced design across all metrics. **Therefore, you should try to be the best on each metric.**

7. Submission Requirements

- Trained model weights, in a specified standard format, along with inference code that accepts a 256x256x3 corrupted image and returns a 256x256x3 denoised image.
- A short technical report (1–2 pages) describing your architecture, training objective, and the degradations used during training.

8. Note

Your final ranking depends entirely on your submitted model's actual performance on the private held-out set. There is no substitute for a genuinely working, well-generalizing implementation that performs well on all the metrics.